CIVICFORGE

AI-Driven Constituency Planning & Crowdsourced Innovation Engine

Complete System Design Specification & Prompt Blueprint for Automated Generation

1. Executive Project Summary

The Official Problem & Challenge

The Track: People's Priorities (AI for Constituency Development Planning)

The Problem Statement: Elected representatives (MPs) receive a massive volume of unorganized development requests across fragmented communication channels (voice notes, messages, text, photos). Simultaneously, they lack an objective, data-driven methodology to prioritize these competing proposals against concrete public infrastructure gaps (such as actual school enrollment, localized travel distances, or health center accessibility), leading to sub-optimal budget allocations.

The Challenge: Build an intelligent platform that ingests multi-format, multilingual input from citizens, analyzes recurring themes, visualizes geographic demand hotspots, and merges this qualitative citizen feedback with quantitative demographic data, public datasets, and infrastructure maps to score and rank optimal actionable development works.

The CivicForge Solution Matrix

CivicForge introduces a **Dual-Engine Framework** that bridges the gap between public grievances and engineering resolution:

- **Data Fusion Priority Engine:** Transforms unstructured citizen requests into verified, quantified structural metrics by cross-checking submissions against regional census and GIS datasets.
- **Crowdsourced Innovation Marketplace:** Opens a technical repository where local developers, technical teams, and startups upload modular open-source solutions. The AI matches localized needs to these existing technical blueprints, transforming basic civic issues into actionable "Civic RFPs" (Request for Proposals).

2. Comprehensive Codebase File Structure

The system utilizes a Monorepo workspace format with decoupled frontend and backend services tailored specifically for rapid deployment.

```
civicforge-workspace/
├── frontend/                  # React.js Client Application
│   ├── public/
│   └── src/
│       ├── assets/            # Static assets and icons
│       ├── components/        # Reusable global UI elements
│       │   ├── common/        # Buttons, Inputs, Loaders, Sidebar, Layout
│       │   ├── citizen/       # Submission forms, Media recorders, Vouch buttons
│       │   ├── developer/     # Project submission forms, Open RFP cards
│       │   └── mp/            # Analytics panels, Heatmaps, Priority lists
│       ├── context/           # React Global State Management (Auth, AppState)
│       ├── hooks/             # Custom React Hooks (useAuth, useMap, useFetch)
│       ├── pages/             # Root View Layouts
│       │   ├── CitizenDashboard.jsx
│       │   ├── DeveloperDashboard.jsx
│       │   └── MPDashboard.jsx
│       ├── services/          # API connectivity client layer (Axios instances)
│       │   ├── api.js
│       │   └── endpoints.js
│       ├── utils/             # Helper utilities, text formatters, math calculators
│       ├── App.jsx
│       ├── index.css          # Tailwind CSS base global configurations
│       └── main.jsx
│
├── backend/                   # Node.js + Express Core Application Server
│   ├── config/                # Database configurations & Environment variable
validation
│   │   ├── db.js
│   │   └── passport.js
│   ├── controllers/           # Route Business Logic Controllers
│   │   ├── authController.js
│   │   ├── issueController.js
│   │   ├── solutionController.js
│   │   └── mpController.js
│   ├── middleware/            # Security, file parsing, and authentication locks
│   │   ├── auth.js
│   │   ├── errorHandler.js
│   │   └── upload.js          # Multer setup for voice/photo handling
│   ├── models/                # MongoDB (Mongoose) Data Collections
│   │   ├── User.js            # Unified User Schema (Citizen, Developer, MP roles)
│   │   ├── Issue.js           # Geo-tagged Citizen issues with processing outputs
```

```
│   │   ├── Solution.js        # Developer technical prototypes and source links
│   │   └── RFP.js             # Automated RFPs created from demand clusters
│   ├── routes/                # Express API endpoint declarations
│   │   ├── auth.js
│   │   ├── issues.js
│   │   ├── solutions.js
│   │   └── mp.js
│   ├── server.js              # Server entry point setup
│   └── package.json
│
└── ai-service/                # Python Data Science & LLM Pipeline Service
    ├── config.py              # Environment variables, API tokens (Google AI Studio)
    ├── main.py                # FastAPI gateway routers
    ├── processing/            # Language and audio analysis modules
    │   ├── audio.py           # Whisper transcription pipeline
    │   └── text_analysis.py   # Gemini API category extraction & stress parsing
    ├── fusion/                # Geospatial and data calculation matrix
    │   ├── spatial_check.py   # GeoPandas proximity logic using local datasets
    │   └── scorer.py          # Priority Score calculation engine
    ├── data/                  # Static geojson/csv reference mock data structures
    │   ├── mockup_schools.json
    │   ├── mockup_clinics.json
    │   └── population_census.csv
    └── requirements.txt       # Dependencies: fastapi, uvicorn, google-genai,
geopandas
```

3. Database Schema Models (MongoDB Collections)

To establish full data integrity between the distinct profiles, the backend utilizes structured relational references within Mongoose.

User Collection (Unified)

```
const UserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['citizen', 'developer', 'mp'], required: true },
  createdAt: { type: Date, default: Date.now }
});
```

Issue Collection (Citizen Influx Data)

```
const IssueSchema = new mongoose.Schema({
  citizen: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  title: { type: String },
  originalInput: { type: String }, // Raw transcribed text if voice
  mediaUrl: { type: String },      // S3/Cloudinary link to audio/photo
  mediaType: { type: String, enum: ['audio', 'image', 'text'], required: true },
  location: {
    type: { type: String, default: 'Point' },
    coordinates: { type: [Number], required: true } // [Longitude, Latitude]
  },
  aiMetadata: {
    translatedText: { type: String },
    category: { type: String },
    subCategory: { type: String },
    stressScore: { type: Number, min: 0, max: 10 },
    infrastructureGapDistanceKM: { type: Number }
  },
  priorityScore: { type: Number, default: 0 },
  status: { type: String, enum: ['Open', 'Linked', 'Funded', 'Resolved'], default:
'Open' },
  linkedSolution: { type: mongoose.Schema.Types.ObjectId, ref: 'Solution' },
  vouchedBy: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  createdAt: { type: Date, default: Date.now }
});
IssueSchema.index({ location: '2dsphere' });
```

Solution Collection (Developer Repository)

```
const SolutionSchema = new mongoose.Schema({
  developer: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  title: { type: String, required: true },
  description: { type: String, required: true },
  githubUrl: { type: String },
  demoUrl: { type: String },
  categories: [{ type: String }], // Matching tags
  vouchersCount: { type: Number, default: 0 },
  createdAt: { type: Date, default: Date.now }
});
```

4. Mathematical Priority Engine Logic

To eliminate subjective bias from infrastructure requests, the system calculates a strict mathematical ranking for every active demand hotspot:

$$Priority\ Score = [\ (C_R \times W_1) + (S_S \times W_2)\ ] \times [\ 1 + (D_{GAP} \times W_3)\ ]$$

Where the variable definitions are set as:

- **C_R (Complaint Recurrence Factor):** Normalized score representing the localized volume of unique citizen complaints logged within a strict 500-meter geo-spatial radius.
- **S_S (Acoustic Stress/Urgency Value):** Sentiment stress scale (0.0 to 10.0) evaluated from text indicators and vocal inflection parameters parsed through the Gemini LLM.
- **D_{GAP} (Infrastructure Deficit Distance):** Computed physical travel distance in kilometers from the specific issue epicenter to the closest functional public amenity matching that category, derived via spatial search over government master facility registries.
- **W_1, W_2, W_3:** Normalization weight variables assigned dynamically to scale structural requirements appropriately across varying geographic regions.

5. Complete Monorepo System Implementation Prompt

> **Operational Notice:** Copy and supply the following block directly into an LLM or automated code generator (like Cursor, Claude, or GitHub Copilot/Codex) to produce the baseline infrastructure files automatically.

```
Act as an expert Full-Stack Software Architect and Software Engineer. Build a working
production-ready prototype codebase for "CivicForge" using Node.js, Express, MongoDB,
React, and FastAPI, matching the structural rules provided below.

Generate the codebase exactly conforming to this execution sequence:

1. Express Server Configurations (backend/server.js)
Setup an Express application supporting CORS, JSON payloads, and dynamic file upload
routes via Multer. Expose endpoints: POST `/api/auth/register`, POST `/api/auth/
login`, POST `/api/issues/submit`, GET `/api/issues/map`, POST `/api/solutions/
register`, GET `/api/mp/prioritization-matrix`.

2. Base Data Model Schemas (backend/models/)
Write pure Mongoose database schemas defining the User Collection, the GeoJSON
```

```
2dsphere-indexed Issue Collection, and the developer Solution Collection according to
the exact field parameters defined in the design file.

3. Complete Python AI Core Router (ai-service/main.py)
Provide a Python FastAPI server running on port 8000. Create a single integrated
handler `/api/analyze-ingestion` that receives multi-format data, calls Google AI
Studio Gemini API via the official `google-genai` framework for categorization and
sentiment analysis, and references local mockup JSON files via GeoPandas to calculate
structural distances.

Include code mapping the exact mathematical Priority Score layout:
Priority Score = ((RecurrenceCount * 1.5) + StressScore) * (1 + (GapDistanceKM * 0.8))

4. MP Analytical Dashboard Layout (frontend/src/pages/MPDashboard.jsx)
Develop a professional, rich client view for the MP interface using React. Fetch data
directly from the express endpoints and map it out cleanly within tables and
geospatial view structures. Display a clean, detailed list sorting issues by their raw
data-driven Priority Score alongside columns for Category, Location coordinates,
Census Gap indicators, and the top matching community-vouched developer project layout
cards.

Code Guidelines: Ensure all imports are completely specified. Write working logic
entirely without utilizing loose truncation comments or placeholders so the system
compiles out-of-the-box.
```